

COMPUTER ARCHITECTURE

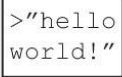
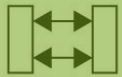
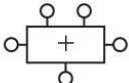
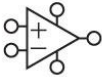


LECTURE 5: Single-cycle Processor

CONTENTS

- > Single-cycle processor
- > Datapath design
 - Methodology
 - Building a simplified RISC-V datapath
- > Control unit design
 - ROM-based approach
 - Combinational logic approach

Introduction

Application Software		Programs
Operating Systems		Device Drivers
Architecture		Instructions Registers
Micro-architecture		Datapaths Controllers
Logic		Adders Memories
Digital Circuits		AND Gates NOT Gates
Analog Circuits		Amplifiers Filters
Devices		Transistors Diodes
Physics		Electrons

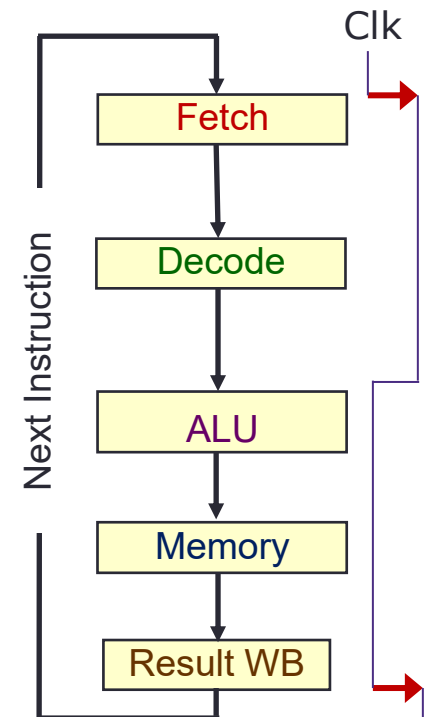
- > Realization of hardware architecture
 - Description: *Block diagrams*
 - μ -architecture realization: Logic
- > Logic (**not covered** in this course)
 - Description: Functional logic units/blocks
 - *Sequential*: outputs depend on current inputs and stored past states
 - *Combinational*: outputs depend on current inputs
- > CPU: processes data and makes decisions
 - **Datapath**: the unit that performs operations
 - **Control**: tells datapath what needs to be done

Single-cycle Processor

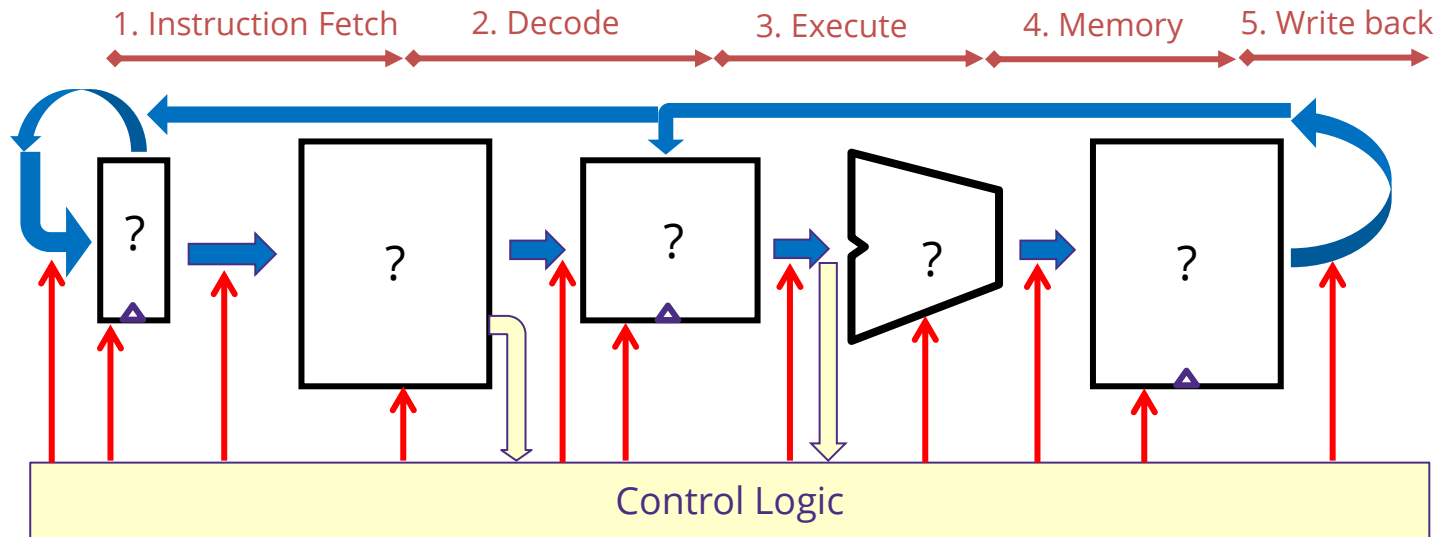
> **Any** instruction is executed in exactly one single clock cycle

- All steps (fetch → decode → ALU → memory → write-back) *are completed within one clock cycle* & execution moves to the next instruction at the start of next cycle

	<code>add t3, t1, t2</code>	<code>lw t3, 20(t1)</code>
Fetch	Read inst. at [PC]	Read inst. at [PC]
Decode & Operand Fetch	○ Addition ○ Read [t1] as <i>opr1</i> ○ Read [t2] as <i>opr2</i>	○ Load word ○ Read [t1] as <i>opr1</i> ○ Use 20 as <i>opr2</i>
ALU	$Result = opr1 + opr2$	$MemAddr = opr1 + opr2$
Memory Access		Use <i>MemAddr</i> to read from memory
Result Write	<i>Result</i> stored in t3	<i>Memory data</i> stored in t3



Design Methodology



- > Break the datapath into simpler stages & build them *incrementally*
 1. **Start small:** Build a minimal RISC-V datapath using a simplified ISA subset
 2. **Analysis:** Analyze required operations and identify requirements at each stage
 3. **Refine:** Sketch a high-level diagram, then detail each component
 4. **Iterate & extend:** Incrementally add & optimize stages for new instruction types

Fetch Stage: Requirements

> Required operations

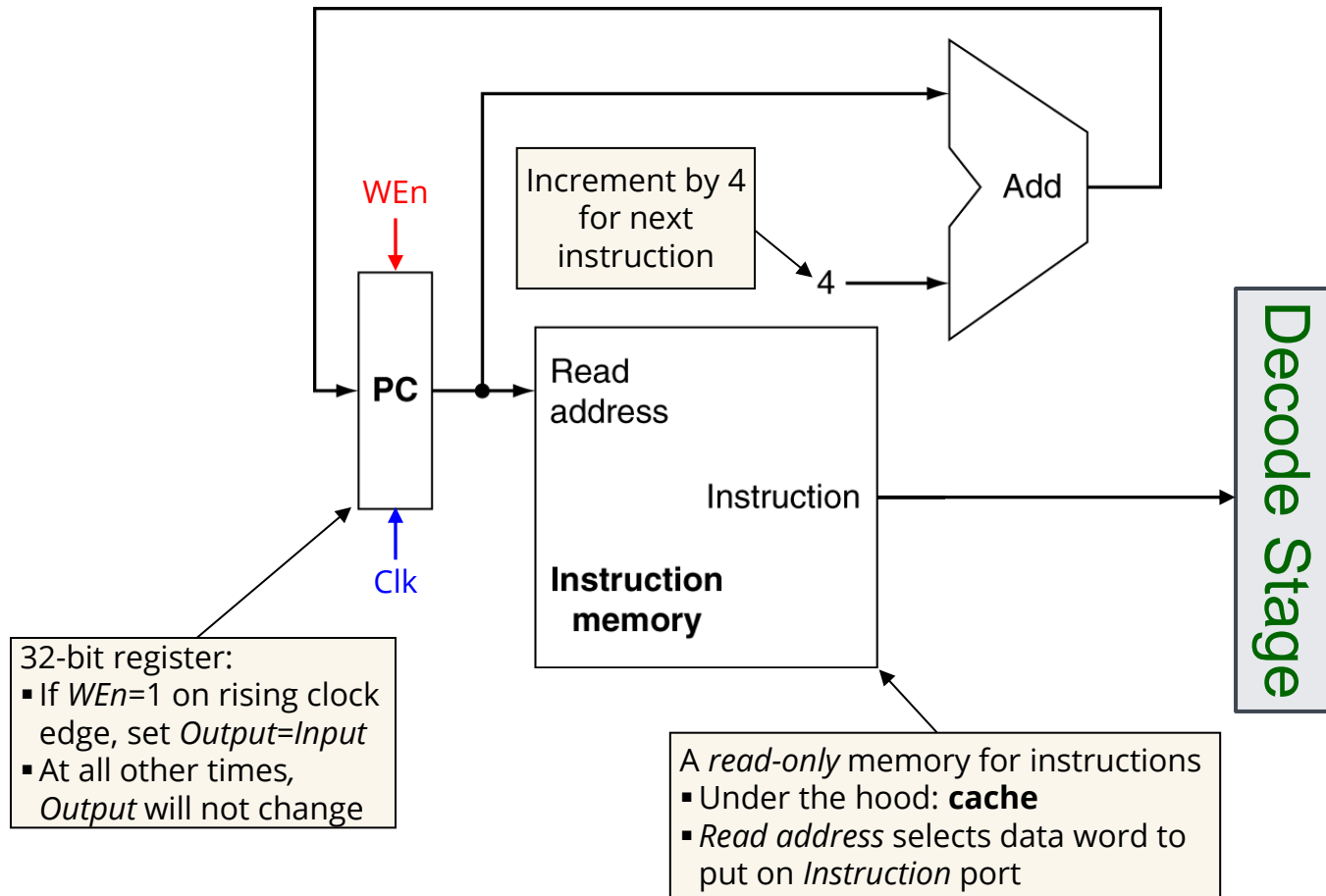
1. Use the **Program Counter** (PC) to fetch the instruction from **memory**
 - PC is implemented as a special register in the processor
2. Increment the PC by 4 to get the address of the next instruction
 - How do we know the next instruction is at PC+4?
 - Note the exception when branch/jump instruction is executed

> Output to the next stage (Decode)

- The instruction to be executed

- | |
|--|
| <ol style="list-style-type: none">1. Fetch2. Decode3. ALU4. Memory5. WB |
|--|

Fetch Stage: Block diagram



Decode Stage: Requirements

> Input from previous stage (Fetch)

- The instruction to be executed

> Required operations

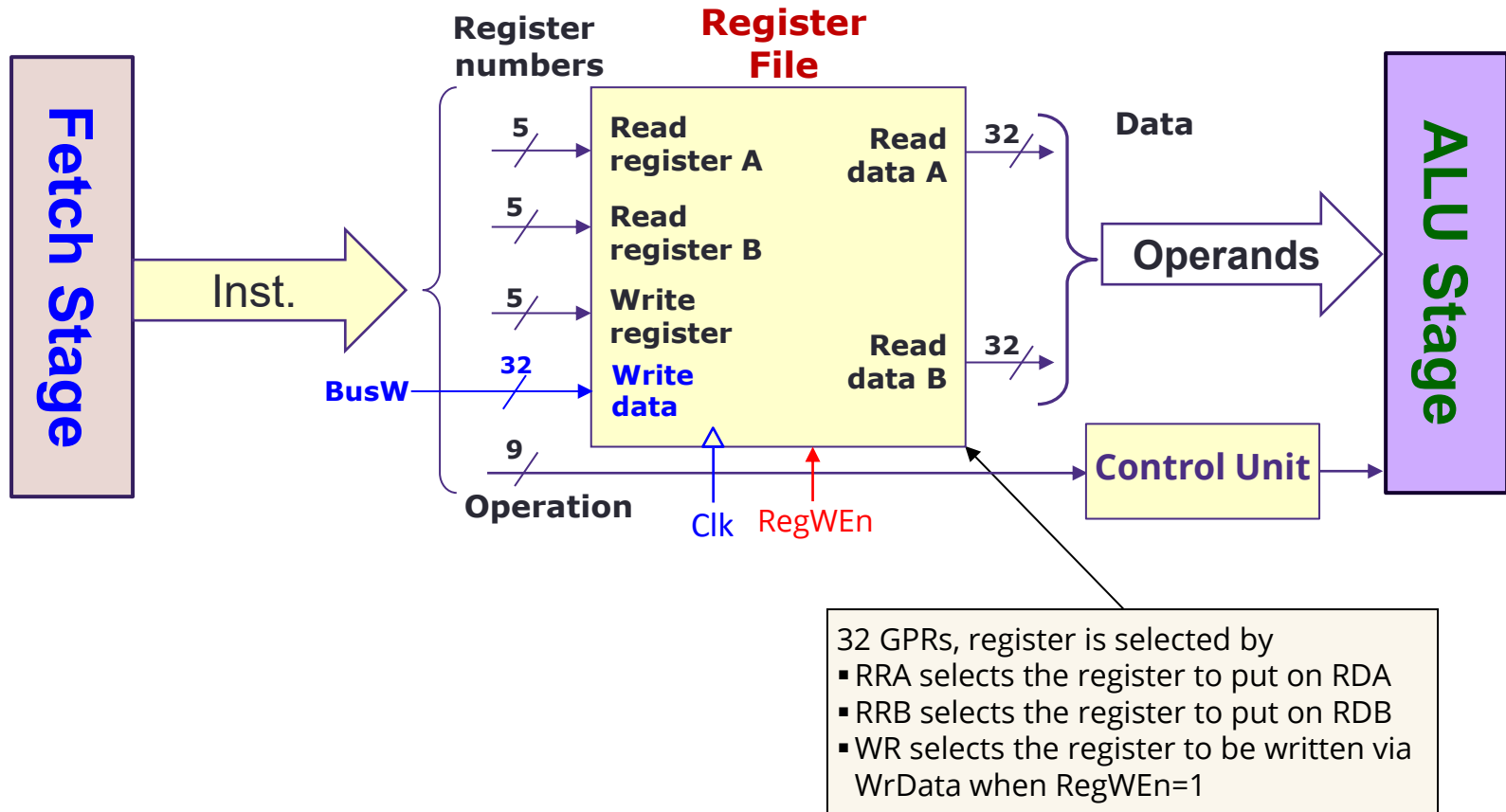
1. Read the **opcode** to determine instruction type and field lengths
2. Read data from general purpose registers (in the register file)
 - Can be two (e.g. `add`), one (e.g. `addi`) or zero (e.g. `auipc`)

> Output to the next stage (ALU)

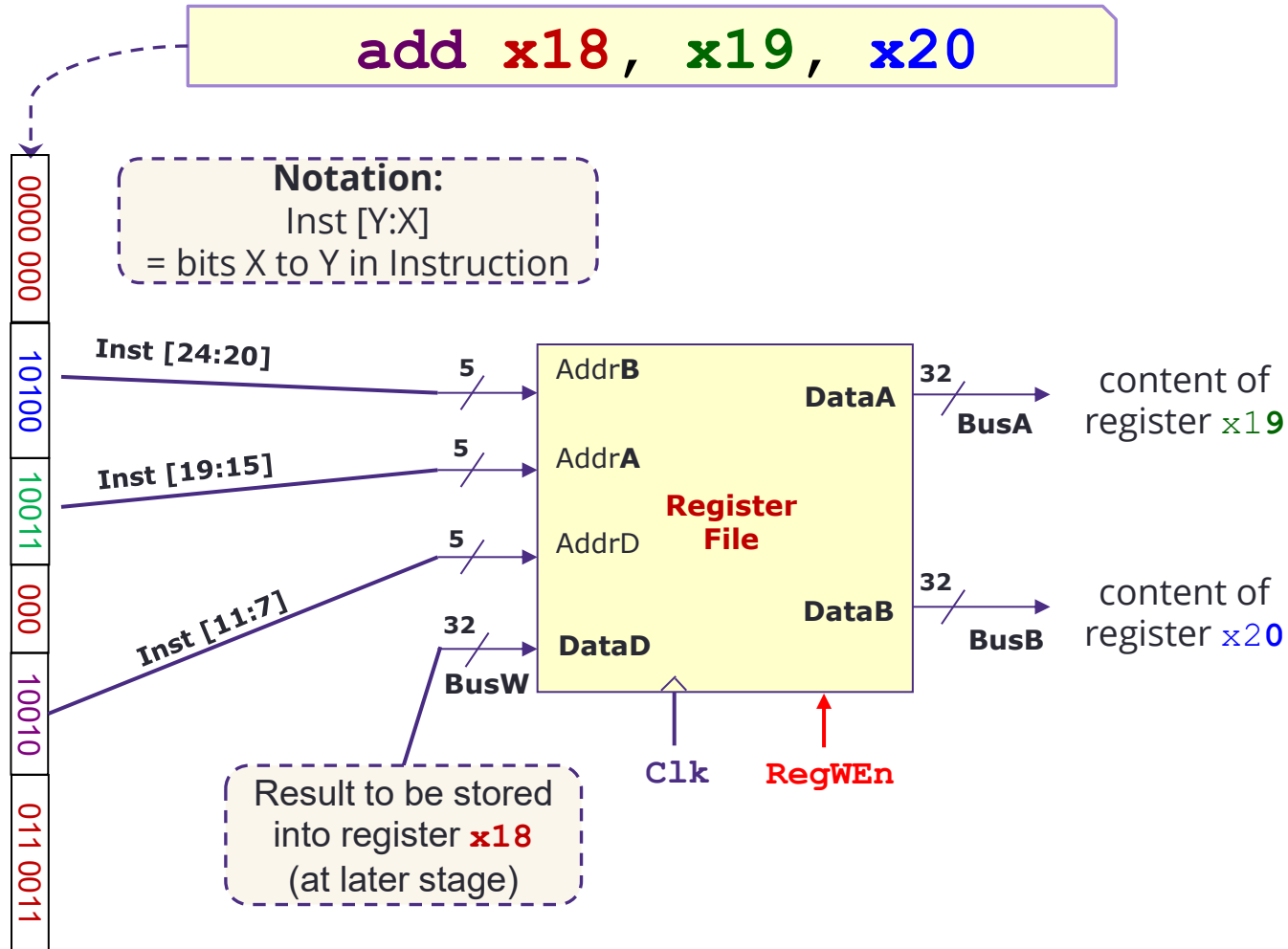
- Operation and the necessary operands

- | |
|------------------|
| 1. Fetch |
| 2. Decode |
| 3. ALU |
| 4. Memory |
| 5. WB |

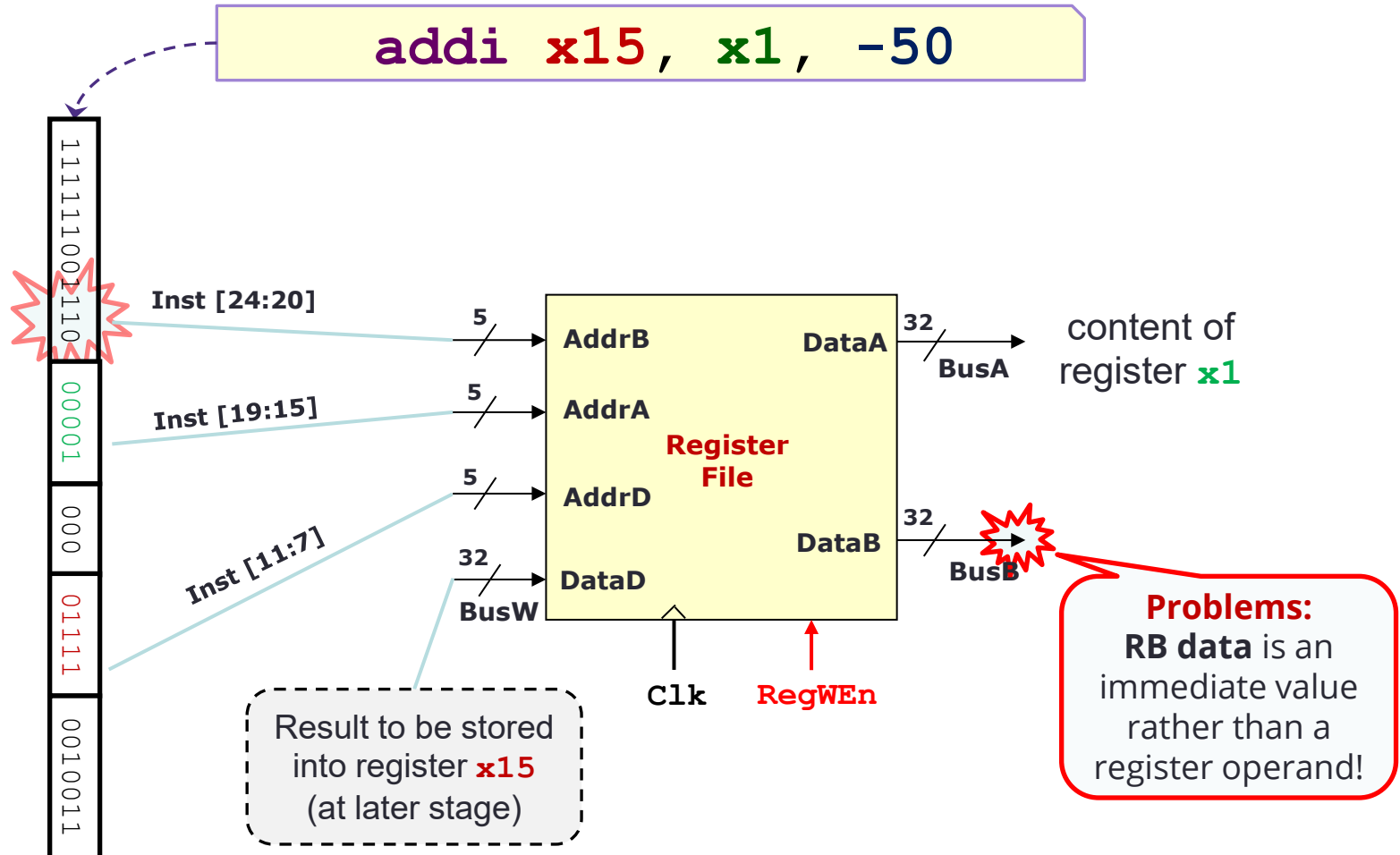
Decode Stage: Block diagram



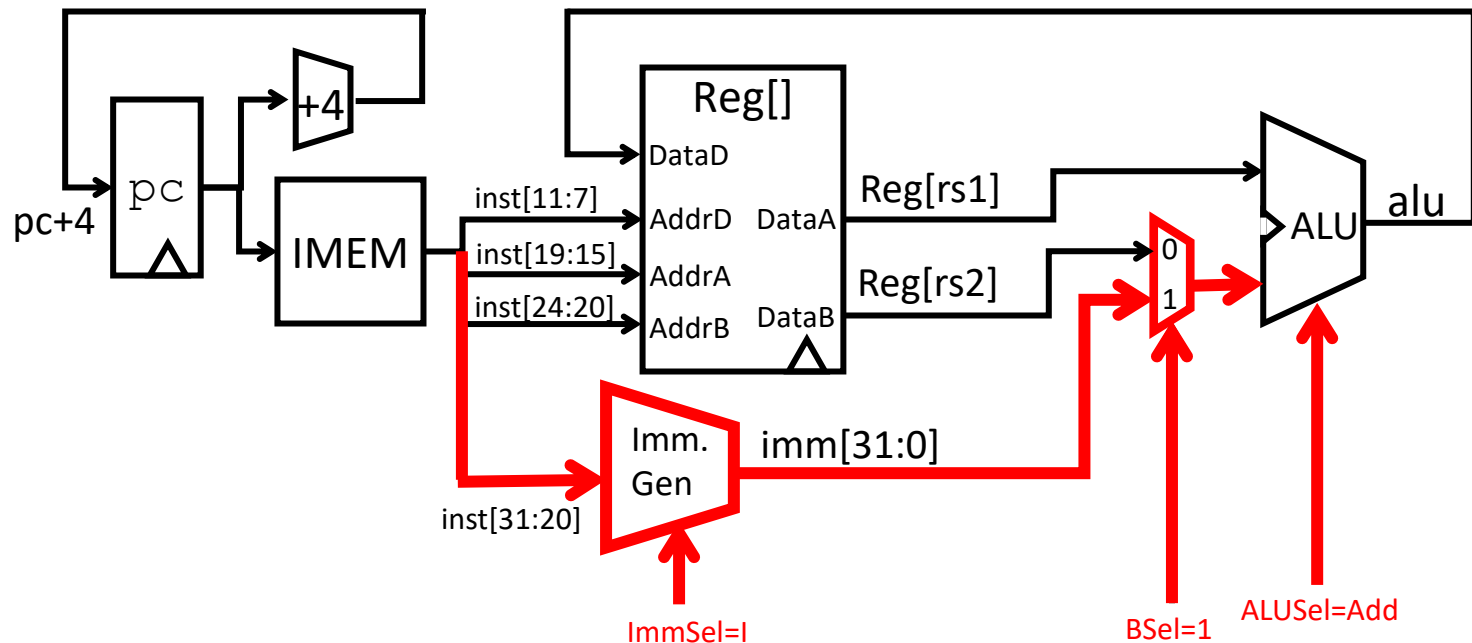
Decode Stage: R-Format Instruction



Decode Stage: I-Format Instruction

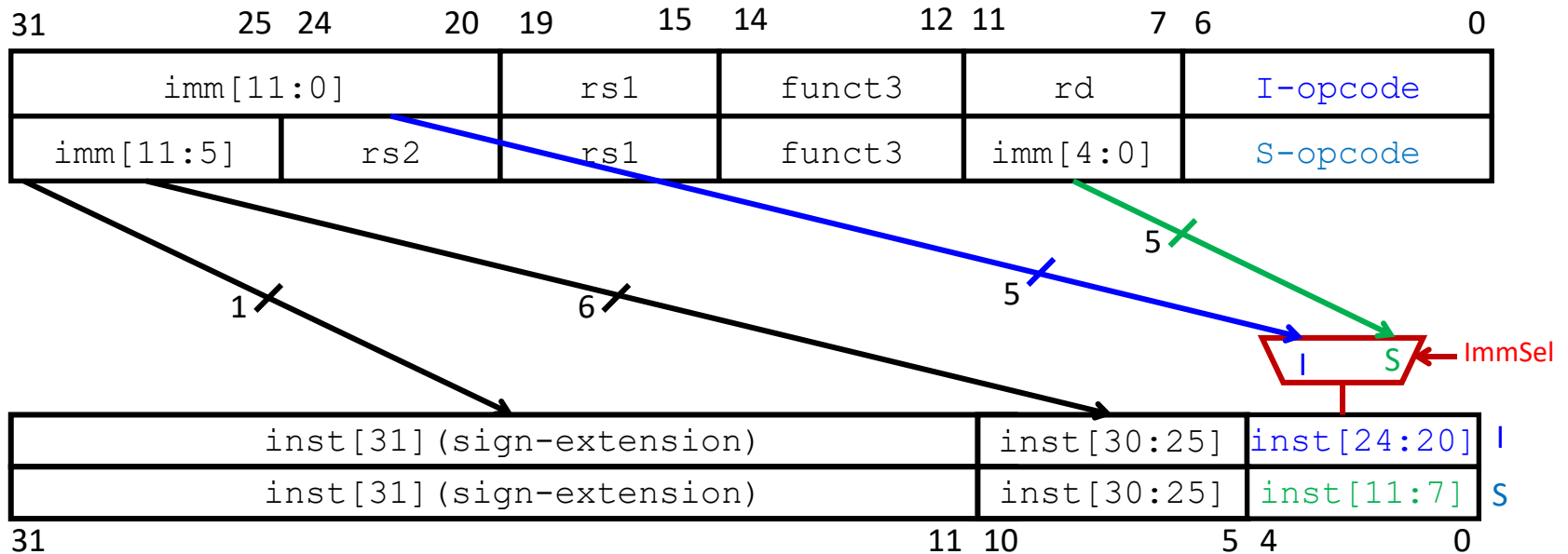


Revised Datapath that Accommodates addi



- > Decoding problem for **addi** is completely solved prior to ALU stage
 - Decoding: copy `inst[31:20]` to low 12 bits of immediate & sign-extended
 - ALU stage: use a MUX prior to the ALU to select `DataB`/immediate operand
 - *Also works for other I-format instructions by changing just the control signal `ALUSel`*

Immediate Generator



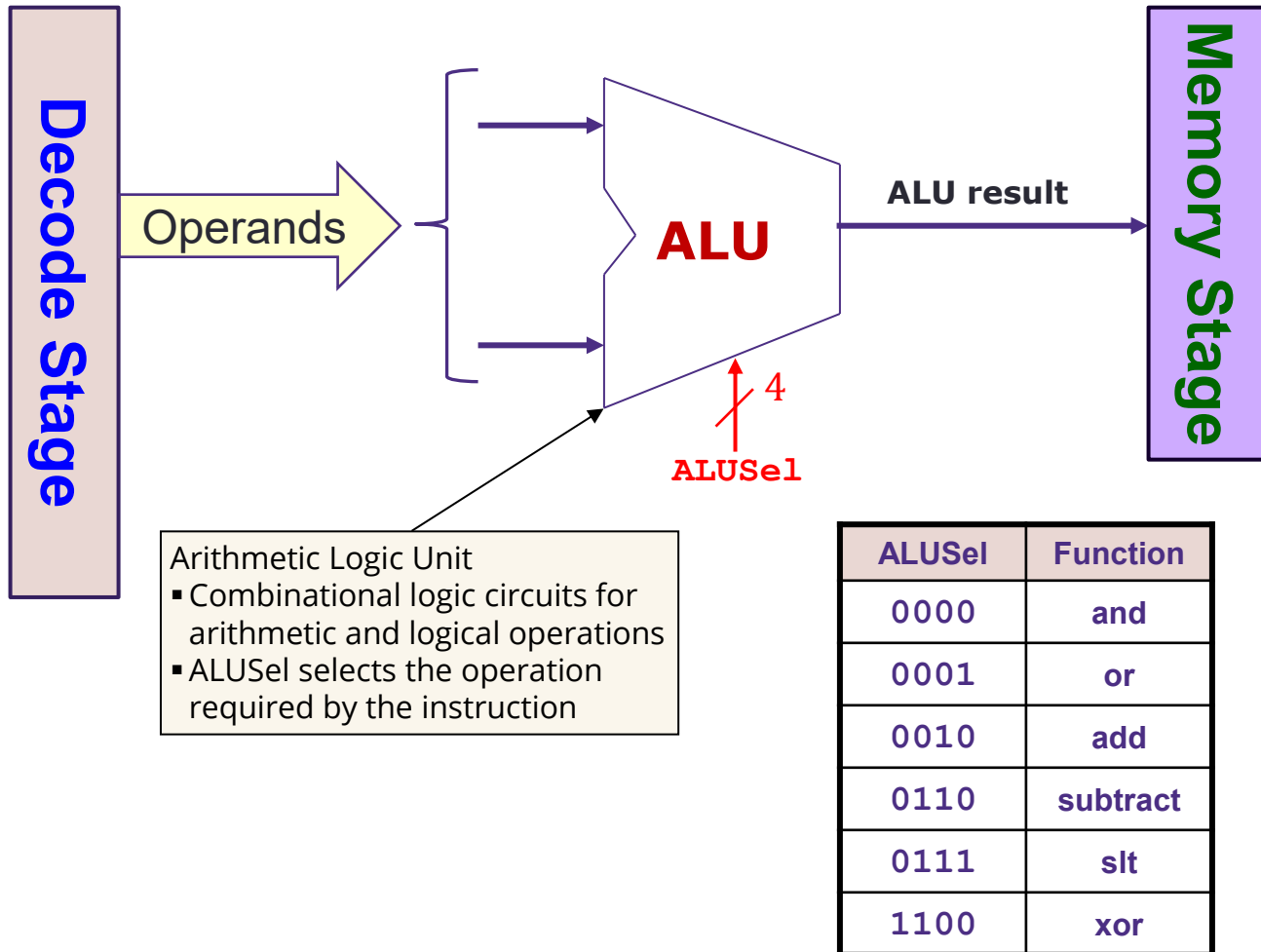
- > immediates are decoded differently for I-type & S-type instructions
 - Need a 5-bit mux to select where the low five bits of `imm` resides in the instruction
 - Other bits in immediate are wired to fixed positions in instruction

ALU (a.k.a. Execution) Stage: Requirements

- > Input from previous stage (Decode)
 - Operation and operand(s)
- > Required operations
 1. Arithmetic (e.g. `add`, `sub`), shifting (e.g. `sll`), logical (e.g. `and`, `or`)
 2. Address calculation for memory operation (e.g. `lw`, `sw`)
 3. Register comparison & target address calculation for branch operations (e.g. `bne`)
- > Output to the next stage (Memory)
 - Calculation result

- | |
|--|
| <ol style="list-style-type: none">1. Fetch2. Decode3. ALU4. Memory5. WB |
|--|

ALU Stage: Block diagram

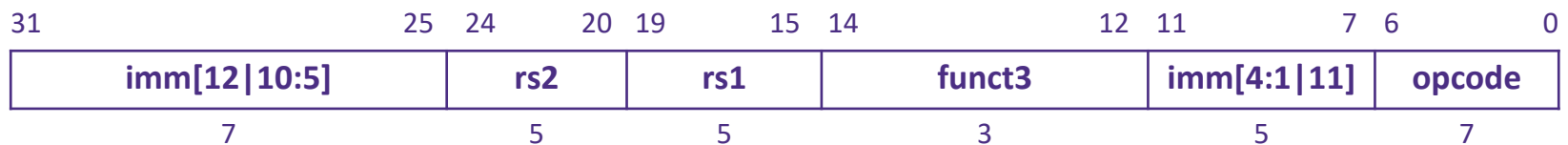


ALU Stage: Implement Branch Instructions

> Branch instructions require dual calculations

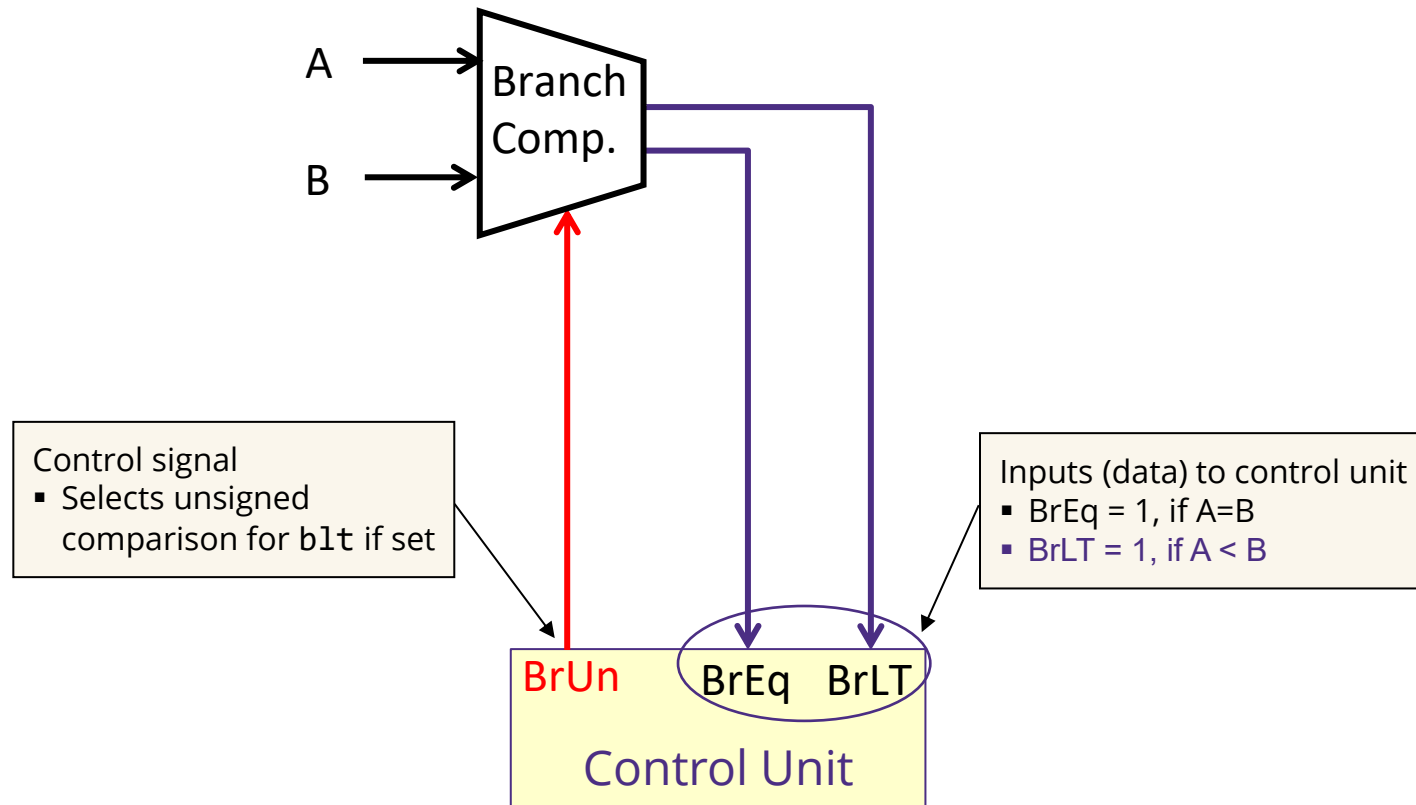
- The branch outcome & the target address
- **Example:** `beq x9, x0, Label`
 - ✓ Compare `x9` & `x0` to determine the branch outcome
 - ✓ Calculate the target address using PC & immediate (if taken)
- Introducing a dedicated comparator to avoid overloading the ALU and simplify timing in a single-cycle design

> B-type: new immediate format

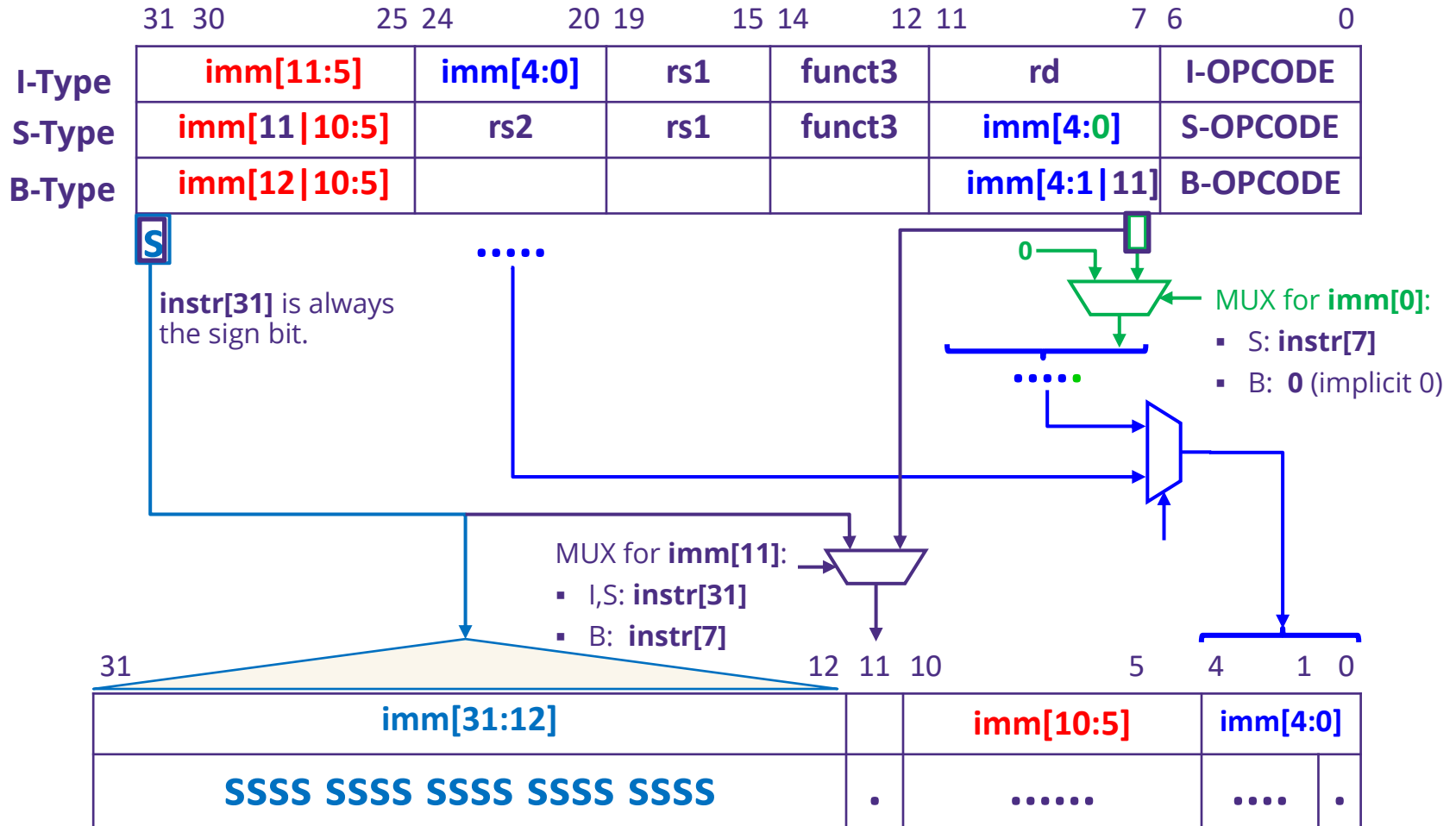


> Must feed the branch target address back to the fetch stage

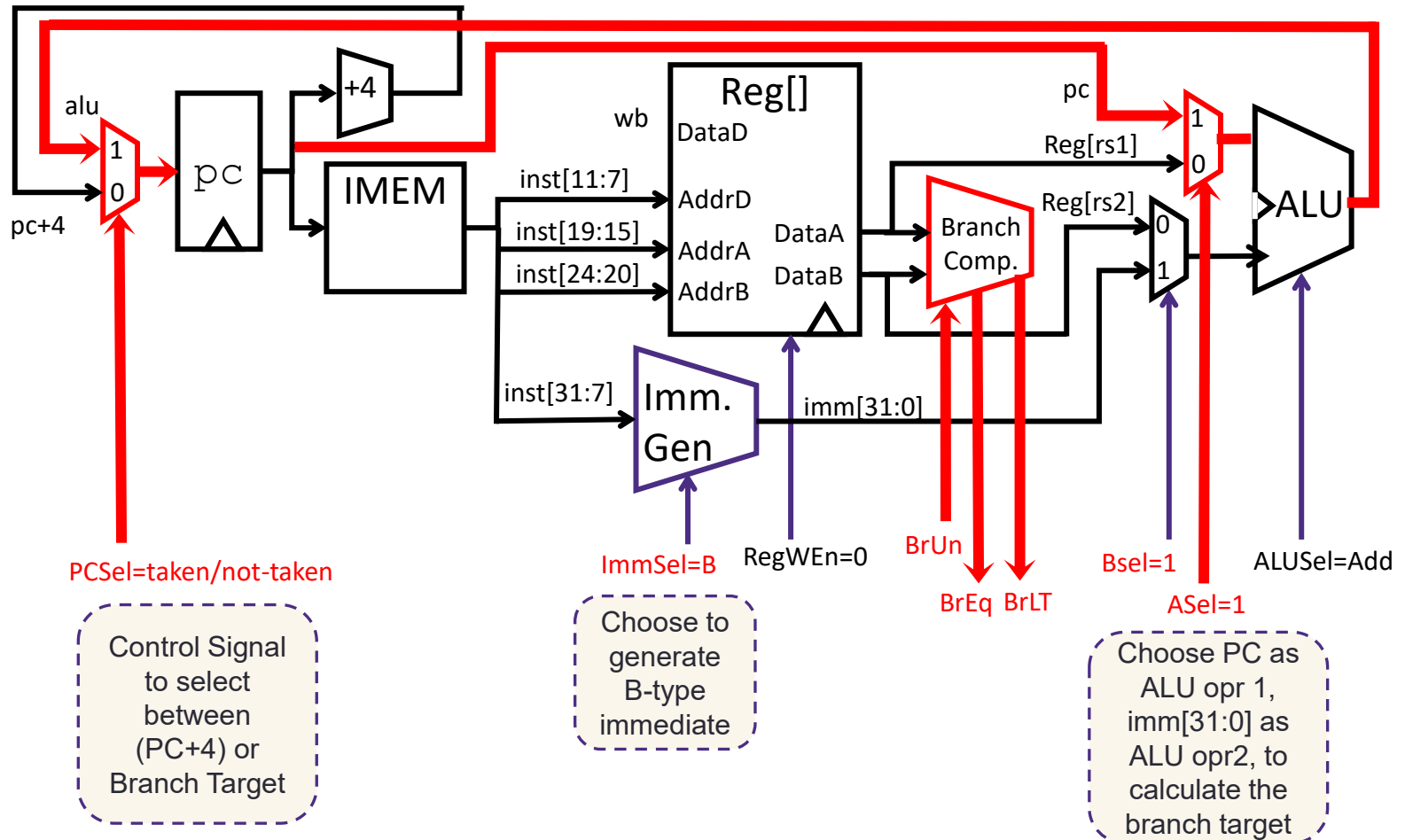
ALU Stage: Branch Comparator



ALU Stage: Immediate Generator Revision



ALU Stage: Adding branch to the datapath

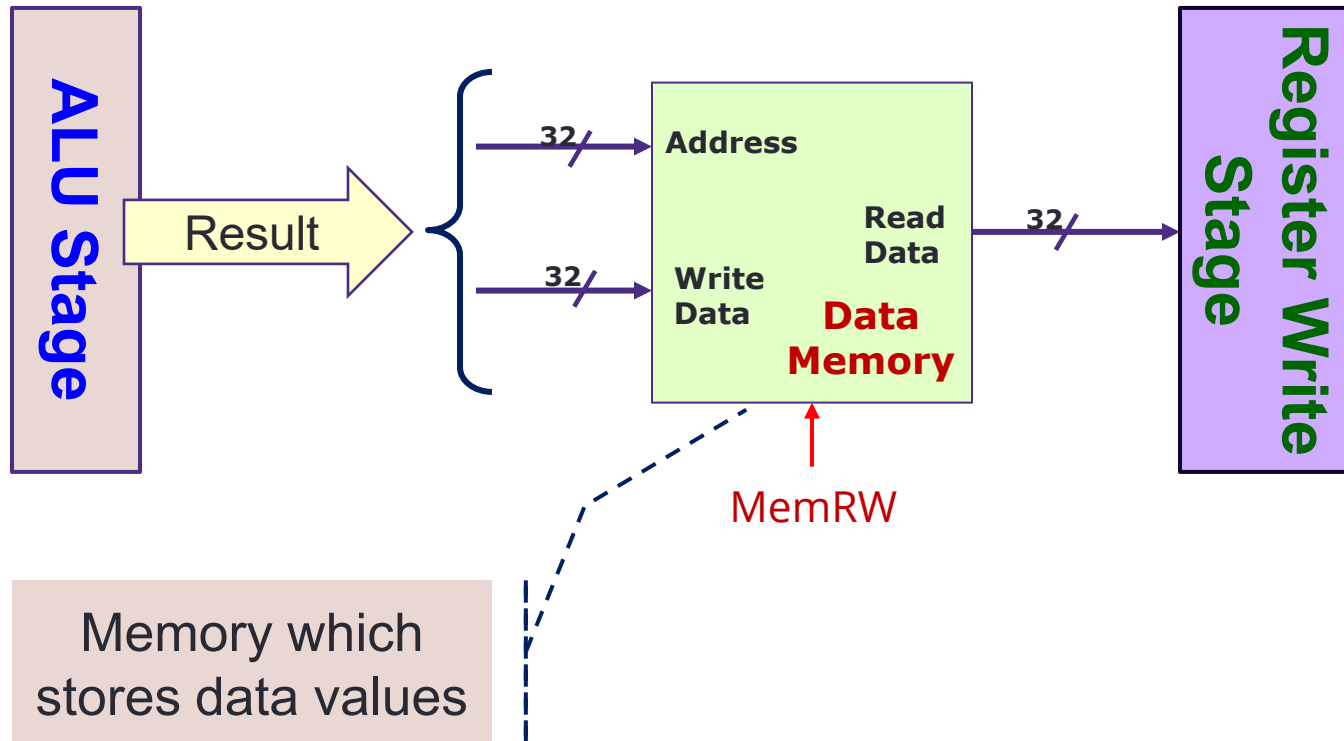


Memory Stage: Requirements

- > Input from previous stage (ALU)
 - Computation result to be used as memory address (if applicable)
- > Required operations
 - **Only** the load and store instructions need to perform operation in this stage
 1. Use memory address calculated by ALU Stage
 2. Read from or write to data memory
 - All other instructions remain **idle**
 - ✓ ALU's result will pass through to be used in Write Back stage (if applicable)
- > Output to the next stage (Write Back)
 - Result to be stored (if applicable)

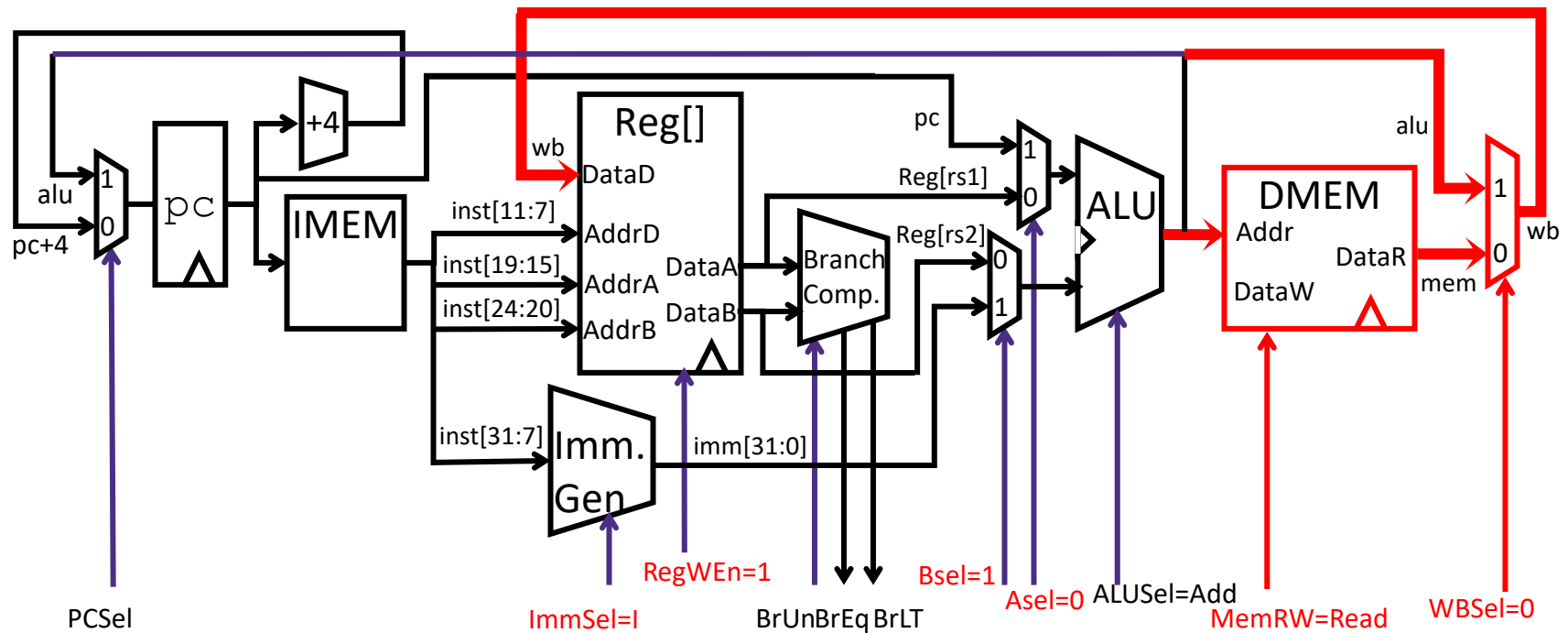
- | | |
|-----------|---------------|
| 1. | Fetch |
| 2. | Decode |
| 3. | ALU |
| 4. | Memory |
| 5. | WB |

Memory Stage: Block diagram



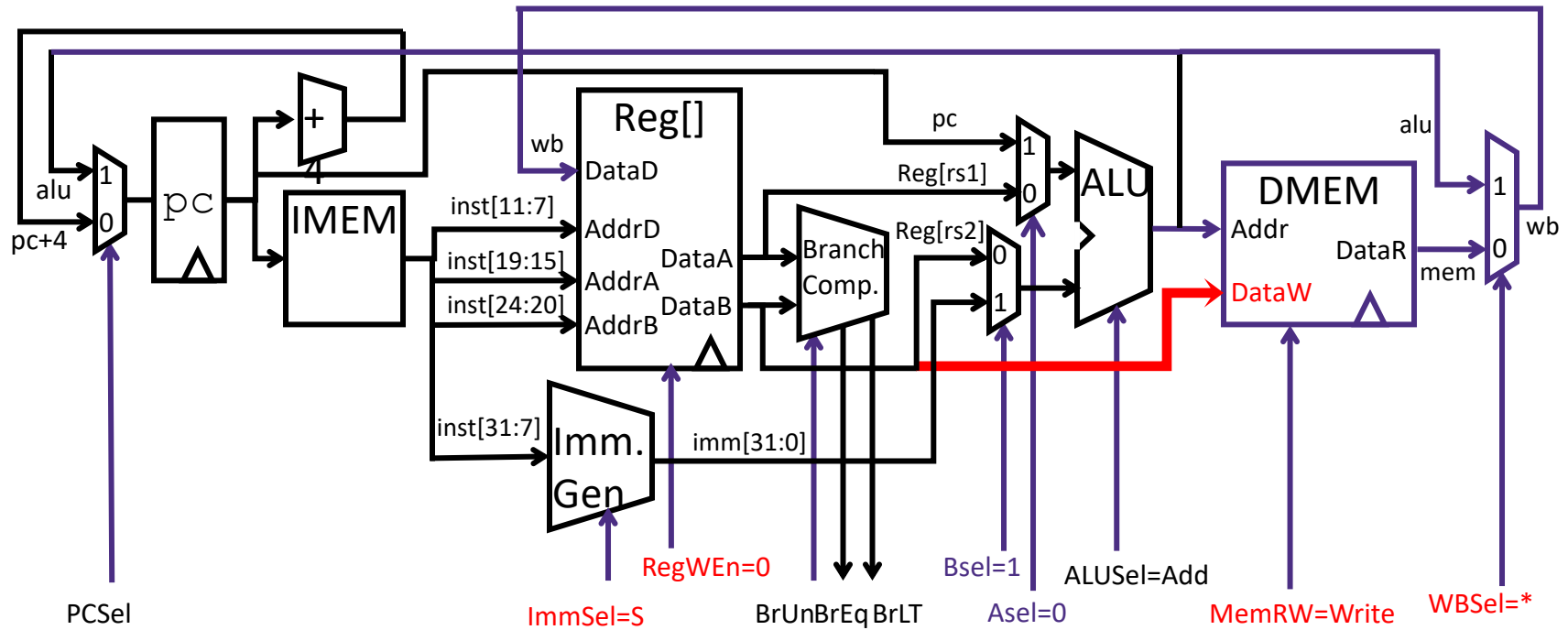
Memory Stage: Adding lw to datapath

lw x14, 8(x2)



Memory Stage: Adding sw to datapath

sw x14, 8(x2)



*= "Don't Care"

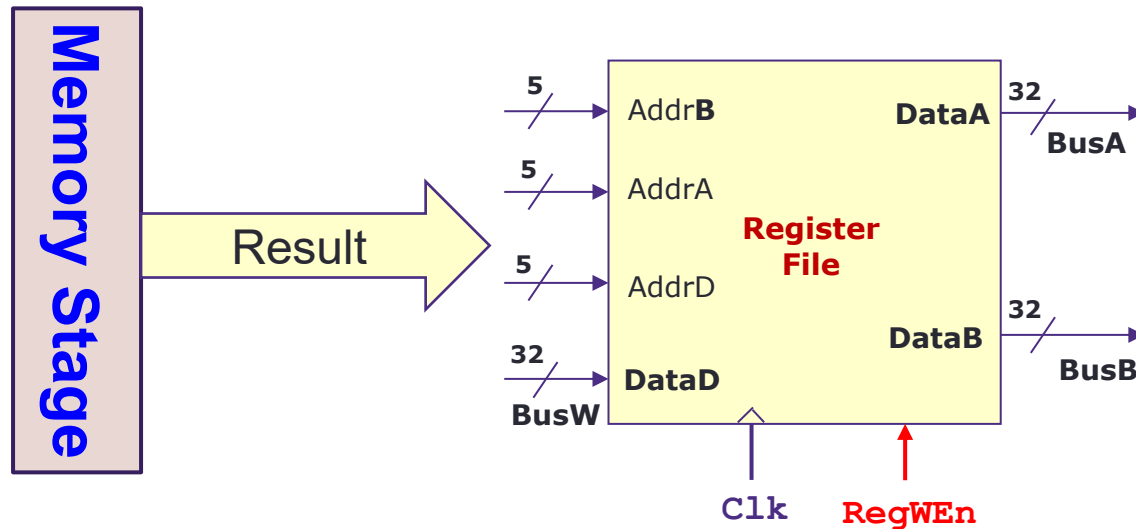
Write Back Stage: Requirements

- > Input from previous stage (ALU)
 - Computation result either from memory or ALU
- > Required operations
 - Most instructions write the result of some computation into a register
 - ✓ Examples: arithmetic, logical, shifts, loads, set-less-than
 - ✓ Need destination register number and computation result
 - Exceptions are stores, branches, jumps
 - ✓ There are no results to be written
 - ✓ These instructions remain idle in this stage

1. Fetch
2. Decode
3. ALU
4. Memory
- 5. WB**

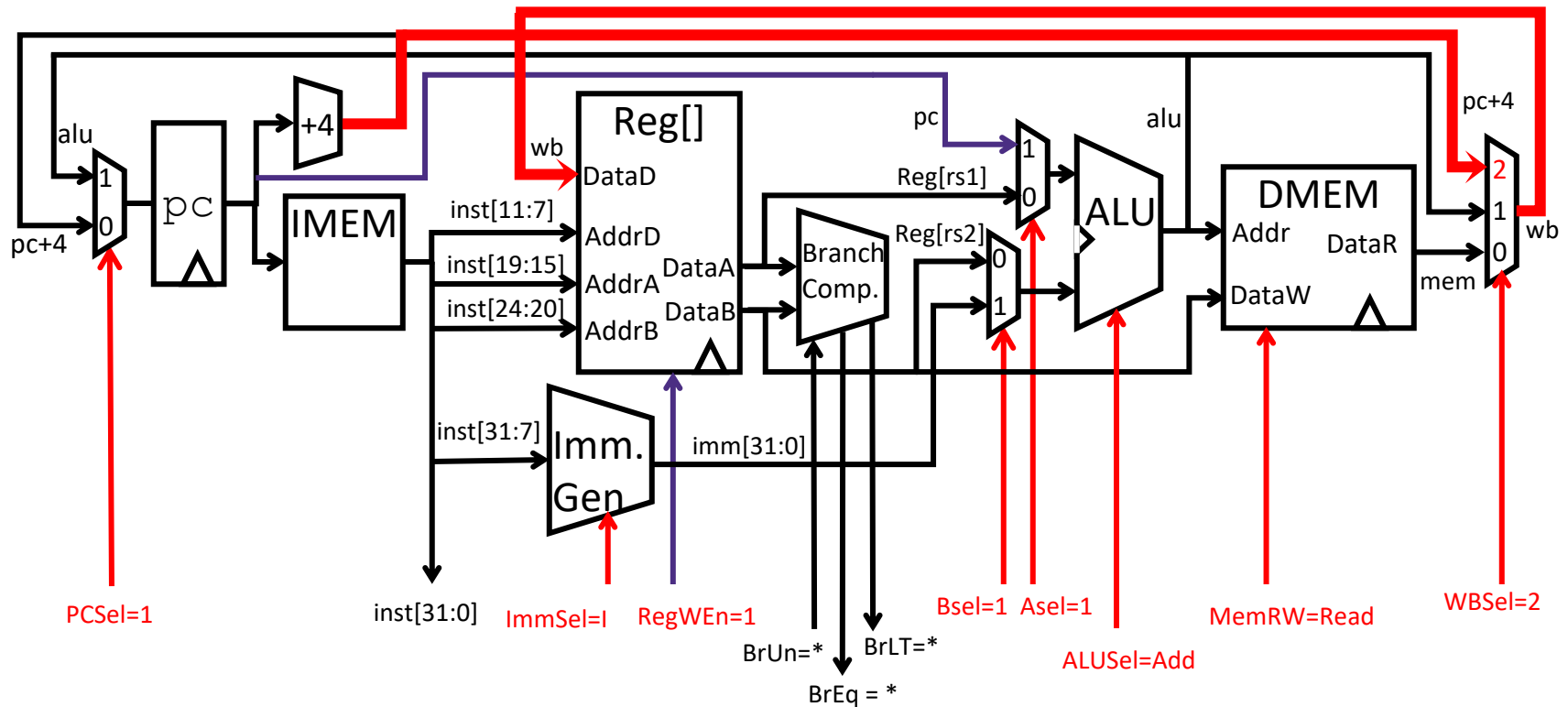
Write Back Stage: Block Diagram

- > Write stage has **no** additional element
 - Basically, just route the correct result into register file
 - The Write Register number had been generated way back in the Decode Stage



Write Back Stage: Adding jalr to datapath

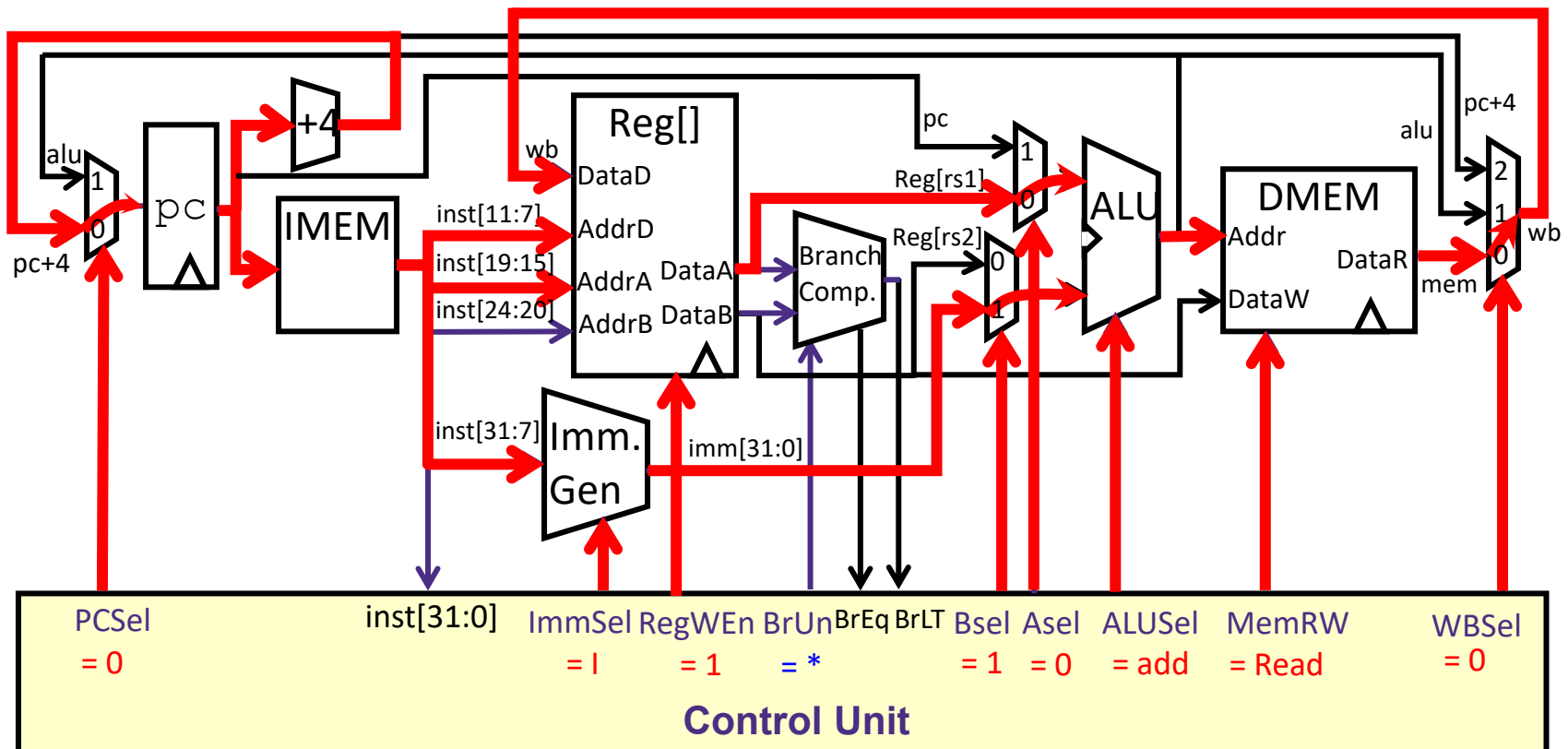
`jalr rd, rs1, imm`



- Enlarging WB MUX to enable PC+4 to be written to Reg[rd]

Control Unit Design: Introduction

- Find the control signals for `lw reg, offset(regbaseptr)`



Summary of RV32I Control Signals

Inst[31:0]	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
add	*	*	+4	*	*	Reg	Reg	Add	Read	1	ALU
sub	*	*	+4	*	*	Reg	Reg	Sub	Read	1	ALU
addi	*	*	+4	I	*	Reg	Imm	Add	Read	1	ALU
lw	*	*	+4	I	*	Reg	Imm	Add	Read	1	Mem
sw	*	*	+4	S	*	Reg	Imm	Add	Write	0	*
beq	0	*	+4	B	*	PC	Imm	Add	Read	0	*
beq	1	*	ALU	B	*	PC	Imm	Add	Read	0	*
bne	0	*	ALU	B	*	PC	Imm	Add	Read	0	*
bne	1	*	+4	B	*	PC	Imm	Add	Read	0	*
blt	*	1	ALU	B	0	PC	Imm	Add	Read	0	*
bltu	*	1	ALU	B	1	PC	Imm	Add	Read	0	*
jalr	*	*	ALU	I	*	Reg	Imm	Add	Read	1	PC+4
jal	*	*	ALU	J	*	PC	Imm	Add	Read	1	PC+4
auipc	*	*	+4	U	*	PC	Imm	Add	Read	1	ALU

➤ Must know the **instruction type** to generate correct control signals

RV32I Instruction Type Decoding

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20:10:11:19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12:10:5]	rs2	rs1	000	imm[4:1:11]	1100011	BEQ
imm[12:10:5]	rs2	rs1	001	imm[4:1:11]	1100011	BNE
imm[12:10:5]	rs2	rs1	100	imm[4:1:11]	1100011	BLT
imm[12:10:5]	rs2	rs1	101	imm[4:1:11]	1100011	BGE
imm[12:10:5]	rs2	rs1	110	imm[4:1:11]	1100011	BLTU
imm[12:10:5]	rs2	rs1	111	imm[4:1:11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

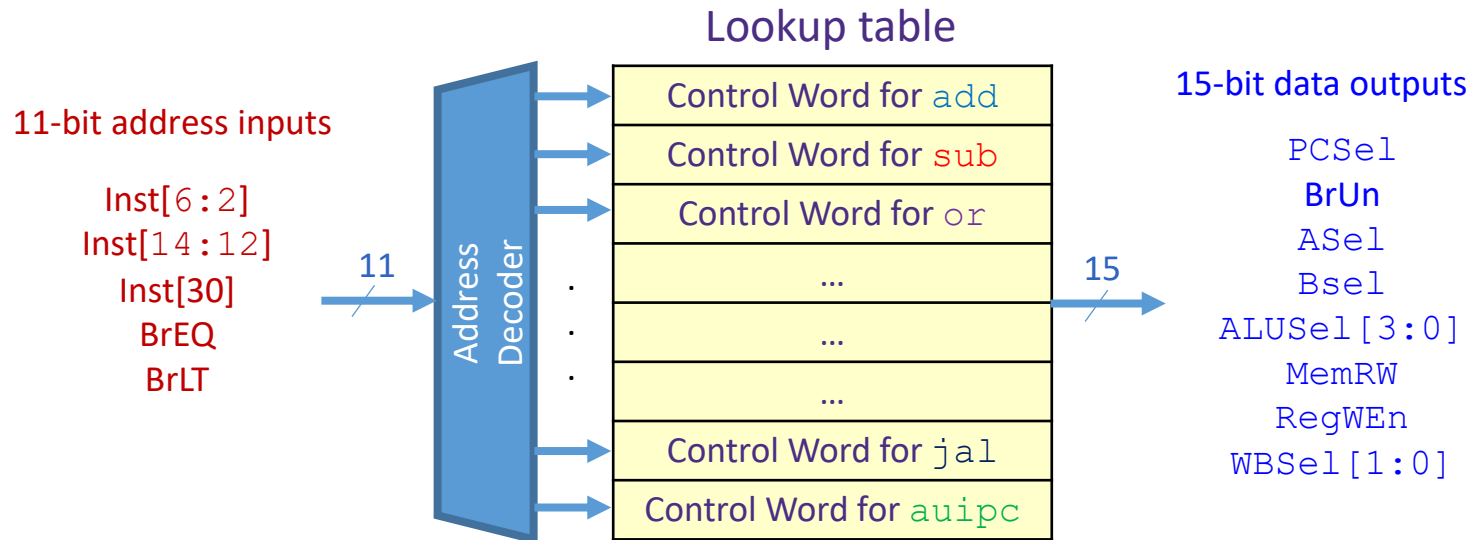
inst[6:2]

inst[14:12]

inst[30]

Instruction types
are encoded by
9 bits only.

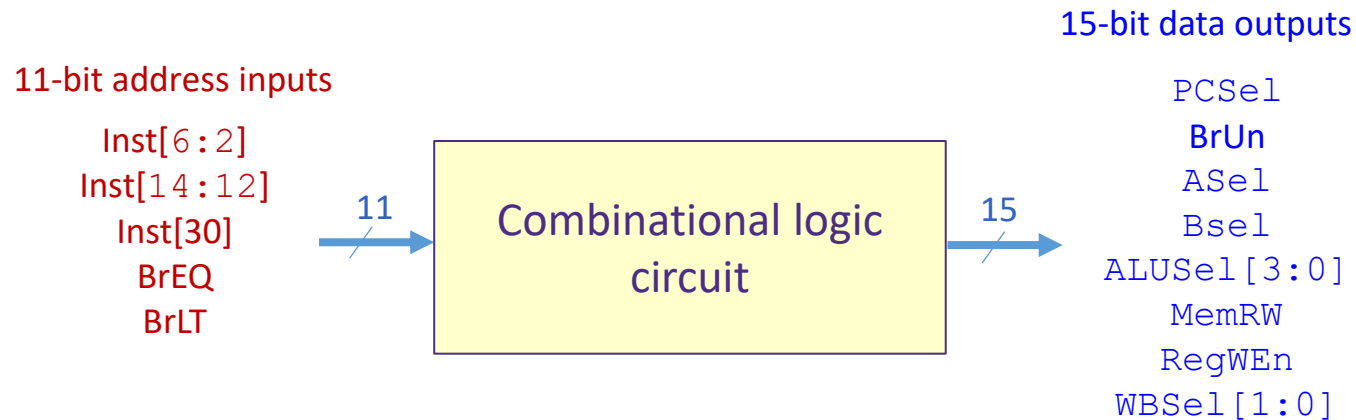
ROM-Based Instruction Type Decoder



> Read-Only Memory (ROM)

- + Regular structure, simplifying the design and manufacturing process.
- + Can be easily reprogrammed when fixing errors or adding instructions
- + Popular when designing control logic manually
- Speed
- Power consumption

Combinational Instruction Type Decoder



- > Use logic gates to directly implement the decoding function
 - + Speed: generally faster than ROM-based implementation
 - + Efficiency (for relatively simple instruction sets)
 - Complexity (for complex instruction sets)
 - Inflexibility: any changes to the instruction set require a redesign of the HW.

NEXT LECTURE

[Flipped class] Pipelined processor

- > Pre-class
 - Study pre-class materials on Canvas
- > In class
 - Reinforcement/enrichment discussion
 - Quizzes
- > Post class
 - Homework
 - Consultation (if needed)